LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

SABIAN KHALIFA IFRIZQIA

109082500135

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

• **Rekursi**

Rekursi adalah teknik dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Rekursi terdiri dari dua bagian utama, yaitu base case (kondisi berhenti) yang digunakan untuk mengakhiri proses pemanggilan fungsi, dan recursive case yaitu bagian yang melakukan pemanggilan ulang fungsi dengan parameter yang lebih kecil atau mendekati kondisi berhenti. Penggunaan rekursi bertujuan untuk menyederhanakan penyelesaian masalah yang dapat dipecah menjadi sub-masalah yang lebih kecil.

• **Deret Fibonacci**

Deret Fibonacci adalah barisan bilangan yang setiap sukunya merupakan hasil penjumlahan dua suku sebelumnya. Deret ini diawali dengan 0 dan 1, kemudian suku berikutnya dihitung dengan rumus:

$$F(n) = F(n-1) + F(n-2)$$

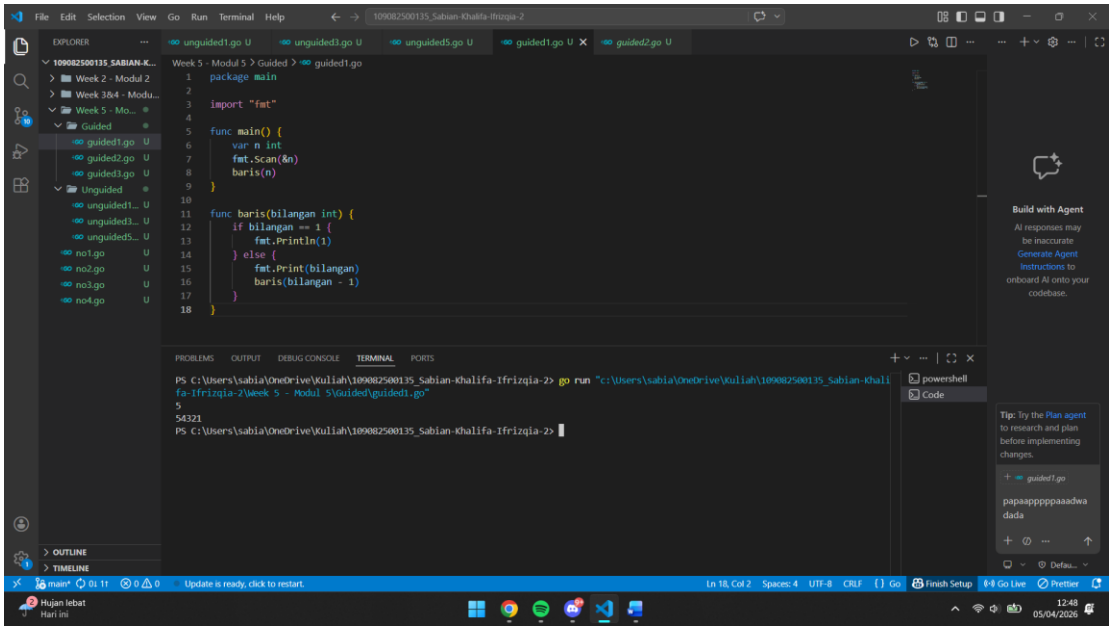
Deret Fibonacci banyak digunakan dalam pemrograman untuk contoh penerapan rekursi karena memiliki hubungan yang jelas antara suatu nilai dengan nilai sebelumnya.

B. Guided

1. [Guided 1]

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Print(bilangan)
        baris(bilangan - 1)
    }
}
```

Output :



Penjelasan :

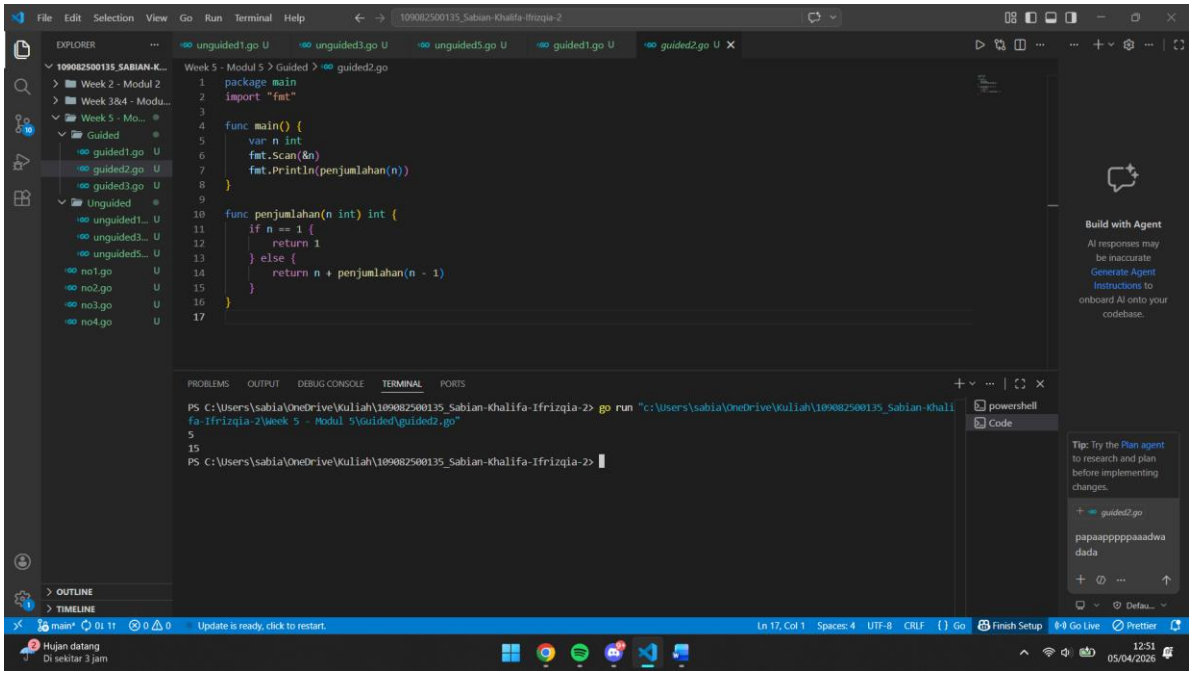
Program ini digunakan untuk menampilkan barisan bilangan secara rekursif berdasarkan nilai input yang diberikan. Program menerima sebuah bilangan bulat sebagai input, kemudian memanggil fungsi baris untuk mencetak deretan bilangan dari nilai tersebut hingga mencapai kondisi tertentu. Fungsi baris bekerja dengan konsep rekursi, yaitu memanggil dirinya sendiri dengan nilai parameter yang terus berkurang. Jika nilai bilangan sama dengan 1, maka program akan mencetak angka 1 sebagai kondisi dasar (base case). Jika tidak, program akan mencetak nilai bilangan saat ini, kemudian memanggil kembali fungsi baris dengan nilai bilangan dikurangi 1.

[Guided 2]

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
```

```
        } else {
            return n + penjumlahan(n - 1)
        }
    }
```

Output :



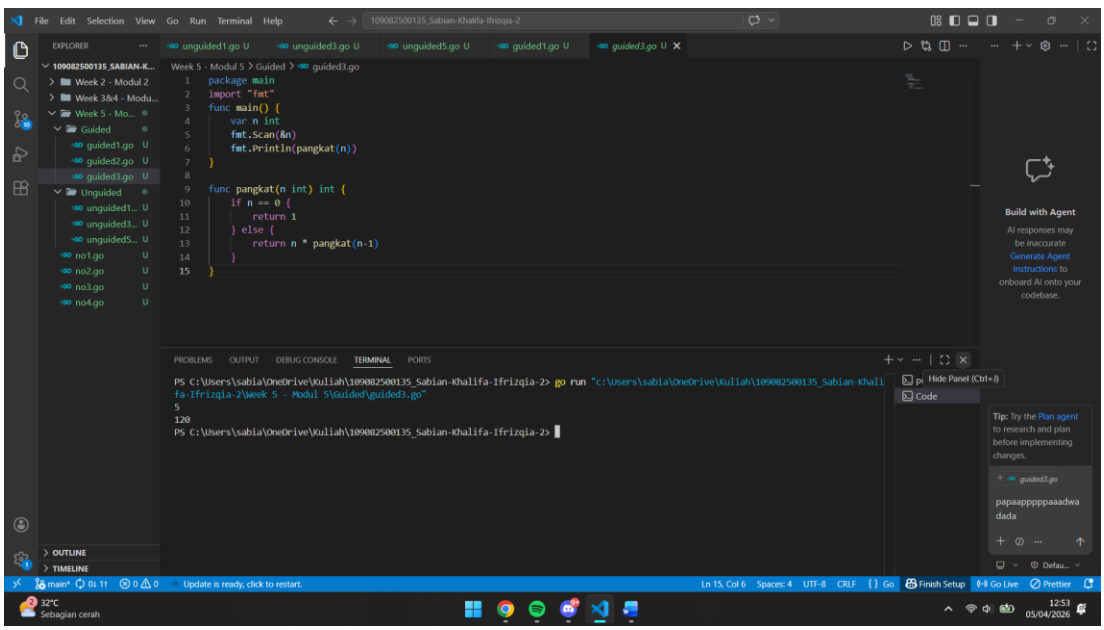
Penjelasan :

Program ini digunakan untuk menghitung jumlah bilangan dari 1 hingga suatu nilai tertentu menggunakan metode rekursif. Program menerima sebuah bilangan bulat sebagai input, kemudian memanggil fungsi penjumlahan untuk melakukan proses perhitungan. Fungsi penjumlahan bekerja dengan konsep rekursi, yaitu memanggil dirinya sendiri dengan nilai parameter yang lebih kecil. Jika nilai n sama dengan 1, maka fungsi akan mengembalikan nilai 1 sebagai kondisi dasar (base case). Jika tidak, fungsi akan mengembalikan hasil penjumlahan antara nilai n dengan hasil dari pemanggilan fungsi penjumlahan(n-1).

[Guided 3]

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return n * pangkat(n-1)
    }
}
```

Output :



Penjelasan :

Program ini digunakan untuk menghitung hasil perkalian berurutan dari suatu bilangan hingga 1 menggunakan metode rekursif. Program menerima sebuah bilangan bulat sebagai input, kemudian memanggil fungsi pangkat untuk melakukan proses perhitungan. Fungsi pangkat bekerja dengan konsep rekursi, yaitu memanggil dirinya sendiri dengan nilai parameter yang lebih kecil. Jika nilai n sama dengan 0, maka fungsi akan mengembalikan nilai 1 sebagai kondisi dasar

(base case). Jika tidak, fungsi akan mengembalikan hasil perkalian antara nilai n dengan hasil dari pemanggilan fungsi pangkat(n-1).

C. Unguided

1. Soal Unguided 1

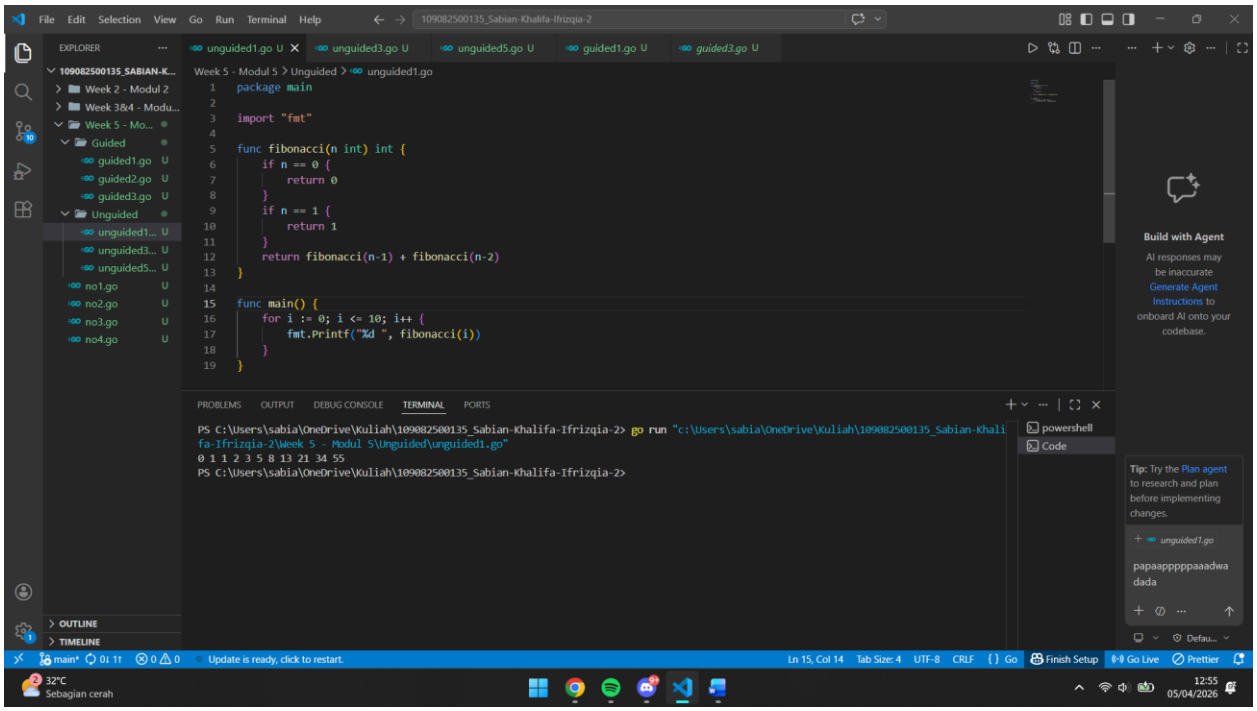
1) Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    for i := 0; i <= 10; i++ {
        fmt.Printf("%d ", fibonacci(i))
    }
}
```

Output :



Penjelasan :

[Program ini digunakan untuk menampilkan deret Fibonacci hingga suku ke-10 menggunakan metode rekursif. Program terdiri dari fungsi fibonacci yang berperan untuk menghitung nilai setiap suku dalam deret, serta fungsi main yang digunakan untuk menampilkan hasilnya.

Fungsi fibonacci menerima sebuah parameter berupa bilangan bulat n. Jika nilai n sama dengan 0, maka fungsi akan mengembalikan nilai 0. Jika nilai n sama dengan 1, maka fungsi akan mengembalikan nilai 1. Kedua kondisi tersebut merupakan **base case** dalam rekursi. Jika nilai n lebih besar dari 1, maka fungsi akan mengembalikan hasil penjumlahan dari fibonacci(n-1) dan fibonacci(n-2).

Pada fungsi main, program menggunakan perulangan dari 0 hingga 10 untuk memanggil fungsi fibonacci pada setiap nilai indeks. Hasil dari setiap pemanggilan fungsi kemudian ditampilkan menggunakan fmt.Printf, sehingga menghasilkan deret Fibonacci secara berurutan.]

2. Soal Unguided 3

   Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan

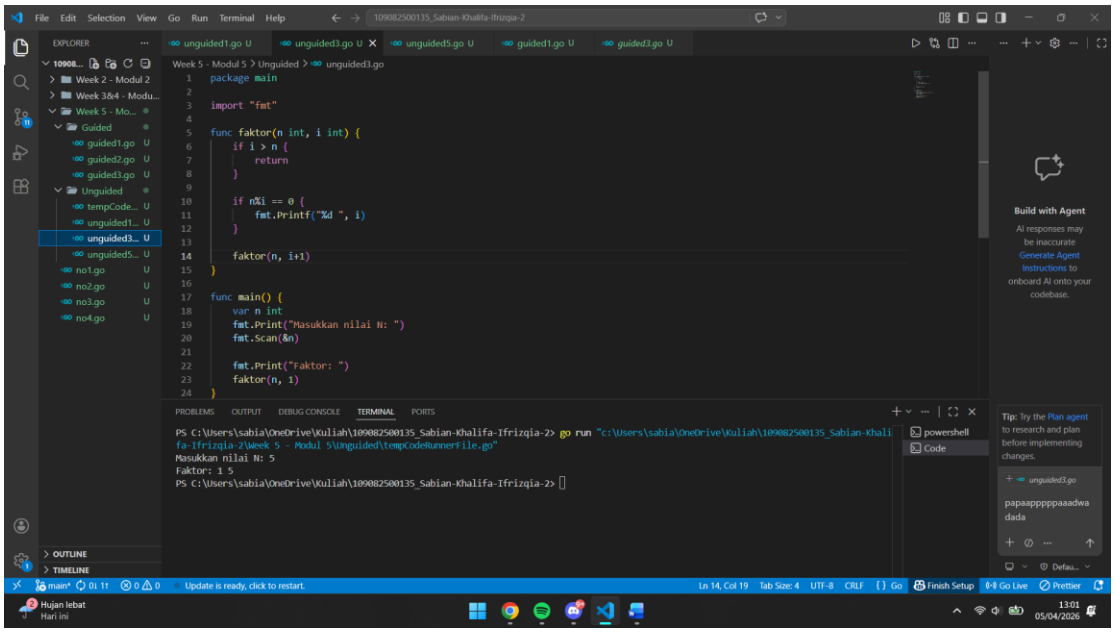
bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya). Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5

Jawaban :

```
package main

import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }

    if n%i == 0 {
        fmt.Printf("%d ", i)
    }

    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)

    fmt.Print("Faktor: ")
    faktor(n, 1)
}
```

Output :



Penjelasan :

[Program ini digunakan untuk menampilkan faktor-faktor dari suatu bilangan menggunakan metode rekursif. Program menerima sebuah bilangan bulat sebagai input, kemudian memanggil fungsi faktor untuk menentukan bilangan-bilangan yang dapat membagi habis nilai tersebut.

Fungsi faktor memiliki dua parameter, yaitu n sebagai bilangan yang akan dicari faktornya dan i sebagai bilangan pembagi yang dimulai dari 1. Jika nilai i lebih besar dari n, maka proses rekursi akan dihentikan. Jika tidak, program akan memeriksa apakah n habis dibagi oleh i. Jika kondisi tersebut terpenuhi, maka nilai i akan dicetak sebagai faktor dari n.

Selanjutnya, fungsi akan memanggil dirinya sendiri dengan nilai i yang ditambahkan 1. Proses ini akan terus berulang hingga seluruh bilangan dari 1 sampai n diperiksa. Dengan demikian, program akan menghasilkan output berupa daftar faktor dari bilangan yang diinputkan secara berurutan.]

3. Soal Unguided 5

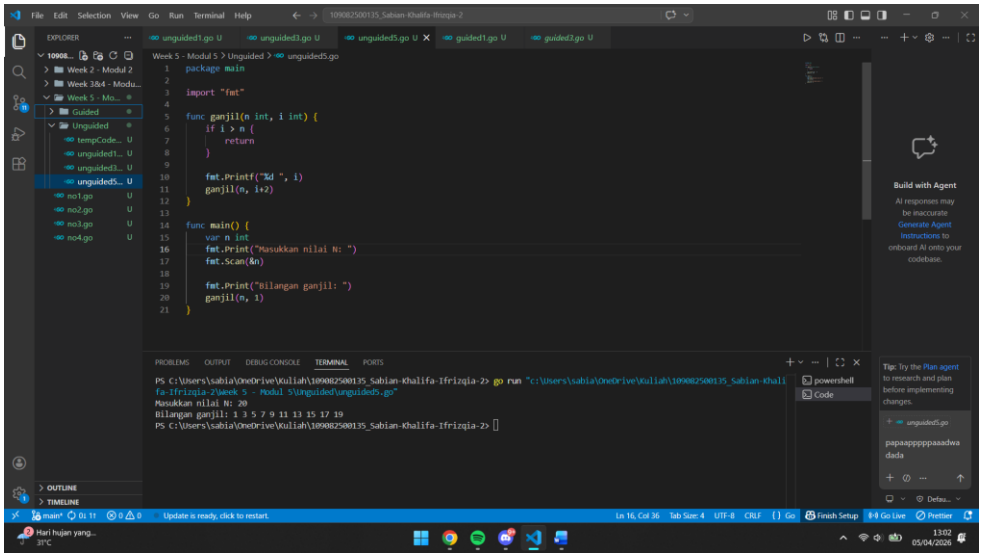
   Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N. Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```go
package main

import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }

    fmt.Printf("%d ", i)
    ganjil(n, i+2)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)

    fmt.Print("Bilangan ganjil: ")
    ganjil(n, 1)
}
```

Output :



Penjelasan :

Program ini digunakan untuk menampilkan barisan bilangan ganjil dari 1 hingga suatu nilai tertentu menggunakan metode rekursif. Program menerima sebuah bilangan bulat sebagai input, kemudian memanggil fungsi ganjil untuk mencetak bilangan ganjil hingga batas yang ditentukan.

Fungsi ganjil memiliki dua parameter, yaitu n sebagai batas akhir bilangan dan i sebagai nilai awal yang dimulai dari 1. Jika nilai i lebih besar dari n, maka proses rekursi akan dihentikan. Jika tidak, program akan mencetak nilai i sebagai bilangan ganjil.

Selanjutnya, fungsi akan memanggil dirinya sendiri dengan nilai i yang ditambahkan 2. Hal ini dilakukan karena bilangan ganjil memiliki selisih 2 antar anggotanya. Proses ini akan terus berulang hingga nilai i melebihi n.

D. Kesimpulan

Dari beberapa program yang telah dibuat dapat dipahami bahwa metode rekursi merupakan teknik pemrograman yang memanfaatkan pemanggilan fungsi secara berulang untuk menyelesaikan suatu permasalahan. Setiap program, seperti deret Fibonacci, penjumlahan bilangan, faktorial, pencarian faktor, dan bilangan ganjil, memiliki pola yang sama yaitu adanya kondisi dasar (base case) sebagai titik berhenti dan proses rekursif yang memanggil fungsi dengan parameter yang lebih kecil. Dengan pendekatan ini, masalah yang kompleks dapat dipecah menjadi bagian-bagian yang lebih sederhana sehingga lebih mudah dipahami dan diimplementasikan.

Namun demikian, penggunaan rekursi memiliki kelemahan dalam hal efisiensi, terutama karena adanya pemanggilan fungsi yang berulang dan penggunaan memori stack yang lebih besar dibandingkan metode iteratif. Hal ini terlihat jelas pada kasus seperti Fibonacci yang melakukan banyak perhitungan ulang. Oleh karena itu, meskipun rekursi sangat baik untuk memahami konsep dasar algoritma dan struktur pemrograman, dalam praktiknya perlu dipertimbangkan penggunaan metode lain yang lebih efisien sesuai dengan kebutuhan program.